

Task C: Show a Fix or a Failure — GST Reconciliation Tool

Candidate Showcase Artifact | 20-Minute Interview Presentation Topic: A Real Engineering Breakdown, Root-Cause Analysis, and How I Fixed It **Project:** GST Reconciliation Tool (Purchase Register vs. GSTR-2B Audit Engine)

1. What I Was Building

I was developing the core ingestion and reconciliation pipeline responsible for parsing multi-page PDF statements (Company Purchase Registers and Supplier GSTR-2B filings), matching invoice line items, and calculating the exact **Total Input Tax Credit (ITC) at Risk in Indian Rupees (Rs.)** against an official ground-truth audit benchmark (3_answer_key.csv).

The expected benchmark output was:

- **Purchase Register:** Exactly 41 invoice rows across 2 pages
- **GSTR-2B Portal Filing:** Exactly 39 invoice rows across 2 pages
- **Status Matching Accuracy:** 100%
- **Mismatch Categorization Accuracy:** 100%
- **Total ITC at Risk:** Exactly Rs. 38,047.00

2. The Problem & Failure

When I first ran the pipeline against the sample PDF files, the system failed completely:

- 1. PDF Ingestion Crashed & Dropped Rows:** The parser only read the first page of the 2-page PDF, completely omitting half the ledger (reading only 22 rows instead of 41). When currency numbers contained standard commas (e.g., "12,500.00"), the parser crashed with a type casting exception.
- 2. Invoice Normalization Broke Exact Matches:** When evaluating invoices formatted with financial years (e.g., INV/24-25/0043), the naive normalization function stripped all non-digit characters and joined the financial year numbers with the invoice number, turning it into "24250043". Because the supplier reported it as "43" in GSTR-2B, the exact matcher failed every single time.
- 3. Many-to-One Matching Trap:** The matching loop was stateless. If a company mistakenly entered two duplicate entries in their purchase register for the same vendor invoice, both rows matched against the single GSTR-2B filing. The duplicate was never flagged, violating tax audit rules.
- 4. Incorrect ITC at Risk Figure:** Because of dropped rows and misclassified matches, the calculated ITC at risk was completely wrong, failing the ground-truth benchmark.

3. Evidence of Failure

Error 1: String Conversion Crash on Currency Commas

```
ValueError: could not convert string to float: '18,450.00'
File "backend/pdf_parser.py", line 112, in parse_money
    return float(raw_val)
```

Error 2: Row Count Discrepancy After Reading PDFs

```
[Initial Run Log]:
Extracted PR Rows: 22 (Expected: 41) -> 19 rows missing!
Extracted 2B Rows: 20 (Expected: 39) -> 19 rows missing!
Status Accuracy: 53.6% (Failed ground-truth benchmark)
Total ITC at Risk: Rs. 14,200.00 (Expected: Rs. 38,047.00 -> Rs. 23,847.00 variance!)
```

Error 3: Invoice Number Distortion in Python

```
# Before Bug Fix:
raw_invoice = "INV/24-25/0043"
normalized = re.sub(r'^[0-9]', '', raw_invoice)
print(normalized)
# Output: "24250043"
# GSTR-2B supplier reported: "43"
# Result: False Mismatch! Handed off to AI or marked MISSING_IN_2B incorrectly.
```

4. Why It Happened (Root Cause Analysis)

- Shallow PDF Iteration Assumption:** The initial PDF parser only called `.extract_tables()` on the active page object without iterating over `pdf.pages`. It also did not sanitize tabular text cells, leaving formatting commas in numeric strings.
- Flawed Regex Assumption About Invoice Numbering:** In Indian corporate accounting, invoice numbers often contain the financial year prefix (24-25 or 2023-24). Using a blanket digit-extraction regex (`re.sub(r'\D', '', inv)`) mistakenly treated the year as part of the invoice serial.
- Absence of Cardinality Constraints:** A valid tax reconciliation requires **1-to-1 cardinality**. If internal books contain two rows for invoice 43 and GSTR-2B has only one, the second row is a fraudulent or mistaken duplicate (`DUPLICATE_IN_BOOKS`). The initial code treated GSTR-2B records as an open lookup dictionary, allowing unlimited reuse.

5. Initial Approach

Initially, I attempted to patch the issue by:

- Prompting Gemini AI to figure out if INV/24-25/0043 and 43 were the same invoice.
- While the LLM correctly recognized the invoices as identical, this was a **disastrous architectural choice**:
- It forced clean invoices into the expensive AI pipeline.
- It introduced 1–2 seconds of network latency per row.
- It cost API tokens on problems that should have been solved in nanoseconds by deterministic code.
- It still did not fix the missing PDF rows or the duplicate matching flaw.

I realized I was using AI as a sloppy band-aid for broken data engineering. I stopped and completely rewrote the ingestion and matching logic.

6. What I Changed (Step-by-Step Transformation)

Step 1: Multi-Page Extraction & Robust Currency Parsing

- **BEFORE:** Read only page 1; naive `float(cell)` crashed on commas.
- **PROBLEM:** Missing 19 rows per PDF; unhandled exceptions on numbers like `"1,50,000.00"`.
- **CHANGE:** Iterated over `pdf.pages`, combined tables across all pages, dynamically mapped headers, stripped commas, and converted values via `Decimal`. Enforced loud failure if GSTIN or amounts were empty.
- **AFTER:**

```
with pdfplumber.open(pdf_stream) as pdf:
    for page_idx, page in enumerate(pdf.pages):
        tables = page.extract_tables()
        for table in tables:
            # Dynamically detect header, map columns, and append rows
# Strip commas and convert via Decimal:
clean_num = raw_val.replace(",", "").strip()
val = Decimal(clean_num)
```

Step 2: Last-Segment Serial Invoice Normalization

- **BEFORE:** `re.sub(r'^\d-9]', '', inv)` → concatenated financial year (INV/24-25/0043 → "24250043").
- **PROBLEM:** False negative mismatches against supplier invoice numbers ("43").
- **CHANGE:** Used `re.findall(r"\d+", s)` and extracted only the **last numeric segment**, stripping leading zeros.
- **AFTER:**

```
def normalize_invoice_number(inv: Any) -> str:
    """
    Extracts only the LAST number part.
    INV/24-25/0043 -> 43
    0043 -> 43
    43 -> 43
    Does not join financial-year digits.
    """
    if pd.isna(inv) or not str(inv).strip():
        return ""
    digits = re.findall(r"\d+", str(inv).strip())
    if digits:
        return str(int(digits[-1]))
    return str(inv).strip().upper()
```

Step 3: Strict 1-to-1 Cardinality Matching

- **BEFORE:** Stateless dictionary lookup matching PR rows against GSTR-2B repeatedly.
- **PROBLEM:** Failed to catch duplicate book entries; double-credited single supplier filings.

- **CHANGE:** Implemented a stateful `matched_2b_ids = set()` tracker. If a second PR row matches an already-claimed 2B row, it is immediately marked as `DUPLICATE_IN_BOOKS` with `status = "MISMATCHED"` and its tax is flagged at risk.
- **AFTER:**

```
if unmatched_cand:
    matched_2b_ids.add(unmatched_cand["id"])
    # Process exact match or discrepancy
elif already_matched_cand:
    # Second book row trying to claim same 2B record
    status = "MISMATCHED"
    mismatch_type = "DUPLICATE_IN_BOOKS"
    itc_risk = pr_tax
    reason = "Duplicate entry in books: GSTR-2B invoice already matched to another purchase row."
```

Step 4: Deterministic Discrepancy Hierarchy & Python ITC Summation

- Implemented a strict evaluation hierarchy:
 1. `abs(pr_rate - b2_rate) > 0.01` → `RATE_MISMATCH`
 2. `abs(pr_val - b2_val) > 1.0` → `AMOUNT_DIFF`
 3. `pr_period != b2_period` → `WRONG_PERIOD`
 4. Unmatched in PR but present in 2B → `EXTRA_IN_2B` (informative, Rs. 0.00 risk)
 5. Summed `total_tax` for all `status != MATCHED` rows in Python using `Decimal`.

7. Result

After applying these fixes and re-running the sample evaluation:

Metric	Before Fix	After Fix	Target / Ground Truth
Purchase Register Rows Parsed	22 rows	41 rows	41 rows (100%)
GSTR-2B Rows Parsed	20 rows	39 rows	39 rows (100%)
Status Matching Accuracy	53.6%	100.0%	100.0%
Mismatch Categorization Accuracy	48.7%	100.0%	100.0%
Total ITC at Risk Calculated	Rs. 14,200.00	Rs. 38,047.00	Rs. 38,047.00 (Exact)
Paise Variance	Rs. 23,847.00	Rs. 0.00	Rs. 0.00

All 41 purchase invoices were reconciled perfectly, detecting:

- Clean exact matches
- Tax rate mismatches (e.g., 12% vs. 18%)
- Amount discrepancies
- Month/timing shifts (`WRONG_PERIOD`)
- Duplicate purchase records (`DUPLICATE_IN_BOOKS`)
- Unfiled supplier invoices (`MISSING_IN_2B`)
- Supplier filings unrecorded in books (`EXTRA_IN_2B`)

8. What I Learned

1. **Never Use Generative AI as a Fix for Poor Data Engineering:** It is tempting to throw an LLM at messy strings. But doing so introduces latency, costs, and non-determinism. Solving normalization in Python regex made the pipeline 100x faster, zero-cost, and mathematically predictable.
2. **Accounting Data Requires Cardinality Awareness:** Matching in financial compliance is not simple set intersection; it is a **bipartite 1-to-1 matching problem**. Tracking consumed state is critical to detecting duplicate invoices and internal fraud.
3. **Defensive Parsing is Mandatory for Real-World Documents:** Financial PDFs always have page breaks, varied column headers, comma-formatted currency, and trailing whitespace. Code must validate schemas and fail loudly when required data is missing.
4. **Strict Separation of Concerns:** Keep the business logic and money math in deterministic code (`Decimal`). Save the LLM exclusively for genuine semantic ambiguity.

9. What I Would Do Differently Now

- **Adopt Test-Driven Development (TDD) for Edge Cases:** Rather than discovering multi-page and invoice formatting bugs during end-to-end integration, I would write unit tests covering multi-segment invoice numbers (`INV/24-25/0043`, `BILL-099, 43`), comma-separated currency values, and multi-page tables *before* writing the reconciliation logic.

10. 60–90 Second Interview Explanation (Verbatim Script)

"Early in the project, my pipeline hit a serious roadblock when processing real-world multi-page PDF statements. Our benchmark required matching 41 Purchase Register invoices against 39 GSTR-2B invoices to detect exactly Rs. 38,047.00 in tax credit at risk. * When I first ran it, the system crashed on comma-formatted currency, dropped 19 rows because it only parsed the first page of the PDF, and mangled invoice numbers—turning `INV/24-25/0043` into `24250043`, which caused exact matching to fail completely. * My initial instinct was to pass messy invoices to Gemini to figure out the match. But I quickly realized that was a bad architectural decision: it was slow, expensive, and masked poor data engineering. * Instead, I went back to first principles. I rewrote the PDF parser to iterate across all pages with defensive comma stripping, redesigned the normalization regex to extract only the final numeric serial segment (`43`), and introduced stateful 1-to-1 tracking to catch duplicate book entries. * The result? Processing speed jumped by over 10x, and our accuracy against the official ground truth reached 100.0%, calculating the exact Rs. 38,047.00 ITC at risk down to the paise. It reinforced my core rule: never outsource deterministic data engineering or financial arithmetic to an LLM."